

Classes and Objects

林致宇

Outline

- Defining a Class
- Constructor
- Object Array
- Object and Pointer
- Friend
- Nested Class

Outline

- Defining a Class
- Constructor
- Object Array
- Object and Pointer
- Friend
- Nested Class

Defining a Class

若沒有聲明 Public· Private
則 Class 內所有成員預設為 Private

```
1  #include<iostream>
2  using namespace std;
3
4  class Car {
5  private:
6      double gas;
7  public:
8      int color;
9      double getGas() { return gas; }
10     void fillGas(double x) { gas += x; }
11 };
```

Cannot Access Outside the Class.
// 例如油箱最大只有 30
// setter, getter
Can Access Outside the Class

```
13 int main(int argc, const char * argv[]) {
14     Car car1;
15
16     car1.color = 5;
17     // car1.gas = 3.0; 期中會考抓錯!
18     car1.fillGas(3.0); (看哪裡會CE)
19
20     cout << car1.color << endl;
21     cout << car1.getGas() << endl;
22 }
```

A member function that is defined inside its class member list is called an *inline member function*.

Member Functions

- Car.h
- Car.cpp
- Main.cpp

Car.h

```
1  class Car {  
2  private:  
3      double gas;  
4  public:  
5      int color;  
6      double getGas();  
7      void fillGas(double x);  
8  };
```

Car.cpp

Define the function outside if you have already declared (but not defined) the member function in the class definition.

```
1  #include "Car.h"
2
3  double Car::getGas() {
4      |    return gas;
5  }
6
7  void Car::fillGas(double x) {
8      |    gas += x;
9  }
```

Main.cpp

```
1  #include<iostream>
2  #include "Car.h"
3  using namespace std;
4
5  int main(int argc, const char * argv[]) {
6      Car car1;
7
8      car1.color = 5;
9      car1.fillGas(3.0);
10
11     cout << car1.color << endl;
12     cout << car1.getGas() << endl;
13 }
```


Static Member Variables (Class Variables)

```
1  #include<iostream>
2  using namespace std;
3
4  class Car {
5  public:
6      static int color;
7  };
8
9  int Car::color = 3;
10
11 int main(int argc, const char * argv[]) {
12     Car car1, car2;
13     cout << car1.color << ", " << car2.color << endl; // 3, 3
14     car1.color = 5; // 更好的方法: Car::color = 5;
15     cout << car1.color << ", " << car2.color << endl; // 5, 5
16 }
```

Static!

所有物件共用一份此變數，
不需要每個變數都單獨維護此變數。

雖然只改了car1的color，但由於color是類別變數(所有物件共用一份)，所以Car2的color也會被改到！

Private Static Member Variables

```
1  #include<iostream>
2  using namespace std;
3
4  class Car {
5  private:
6      static double gas;
7  public:
8      static int color;
9  };
10
11 double Car::gas = 0.0;
12 int Car::color = 3;
13
14 int main(int argc, const char * argv[]) {
15     Car car1, car2;
16     cout << car1.color << ", " << car2.color << endl; // 3 , 3
17     car1.color = 5;
18     cout << car1.color << ", " << car2.color << endl; // 5 , 5
19 }
```

Static Member Methods (Class Methods)

Static member functions can only access static member variables.

```
4  class Car {
5  public:
6      static int sv;
7      int color;
8      static void printVariables() {
9          cout << sv << endl;
10         // cout << color << endl; ← Not Allow!
11     }
12 };
13 int Car::sv = 3;
14 int main(int argc, const char * argv[]) {
15     Car::printVariables();
16 }
```

Static Member function
不能使用 unstatic member variables. 会 CE

Outline

- Defining a Class
- Constructor
- Object and Pointer
- Friend
- Nested Class

Constructor

建構函式必須與類別同名，
且不能有任何回傳值

建構子的 function name 就是 Class name,
且沒有回傳值!

```
4  class Car {  
5      private:  
6          double gas;  
7      public:  
8          Car(double gas) { this->gas = gas; }  
9          double getGas() { return gas; }  
10         void fillGas(double x) { gas += x; }  
11     };  
12  
13     int main(int argc, const char * argv[]) {  
14         Car car1 {2.0}; → 也可以寫成 Car car1(2.0);  
15         car1.fillGas(3.0);  
16         cout << car1.getGas() << endl; // 5  
17     }
```

Initialize Public Member Variables (w/o Constructor)

不使用 建構子初使化 Member variables (不考)

```
4  class Point {
5  public:
6      int x, y;
7      void showPoint() { cout << "("
8          << x << ", " << y << ")" << endl; }
9  };
10
11  int main(int argc, const char * argv[]) {
12      Point p1 = {6, 8};
13      p1.showPoint();
14  }
```

Initialize Member Variables

```
4  class Point {  
5  private:  
6      int x, y;  
7  public:  
8      Point(int x, int y): x {x}, y {y} {};  
9      void showPoint() { cout << "("  
10         << x << ", " << y << ")" << endl; }  
11 };  
12  
13 int main(int argc, const char * argv[]) {  
14     Point p1(6, 8);  
15     p1.showPoint();  
16 }
```

另-種寫法

Point (int x, int y){
this→x = x;
this→y = y;
}

Counting the number of objects

```
4  class Car {
5  public:
6      static int counter;
7      Car() { counter++; }
8  };
9
10 int Car::counter = 0;
11
12 int main(int argc, const char * argv[]) {
13     Car car1, car2;
14     cout << Car::counter << endl; // 2
15 }
```

沒有參數的建構子，就是預設建構子

→ 因此只要產生物件，Constructor 就會被呼叫

Constructor

若自行撰寫建構子,
則預設建構子失效!

CE!

```
4  class Car {  
5  private:  
6      double gas;  
7  public:  
8      Car(double g) { gas = g; }  
9      double getGas() { return gas; }  
10     void fillGas(double x) { gas += x; }  
11 };  
12  
13 int main(int argc, const char * argv[]) {  
14     Car car1;  
15 }
```

10_05_Constructor2.cpp:14:9: **error**: no matching constructor for initialization of 'Car'
Car car1;
^

default

Overloading!

```
4  class Car {
5  private:
6      double gas;
7  public:
8      Car(double g) { gas = g; }
9      Car() = default; ← 多載來修正 前頁投影片CE 的問題!
10     double getGas() { return gas; }
11     void fillGas(double x) { gas += x; }
12 };
13
14 int main(int argc, const char * argv[]) {
15     Car car1;
16 }
```

→ 也可寫成 Car() {}

Default Values

如果只是要初始化成員變數，不一定需要使用到多載 (Overloading)

```
4  class Time {
5  public:
6      int hour, min, sec;
7      Time(int, int, int);
8      void show() {
9          cout << hour << ", " << min << ", " << sec << endl;
10     }
11 };
12
13 Time::Time(int h, int m = 1, int s = 1) {
14     this->hour = h;
15     this->min = m;
16     this->sec = s;
17 }
18
19 int main(int argc, const char * argv[]) {
20     Time t1(12); Time t2(6, 8); Time t3(1, 6, 8);
21     t1.show(); t2.show(); t3.show();
22     //12,1,1    //6,8,1    //1,6,8
}
```

Single Parameter

```
4 class Car {
5 private:
6     double gas;
7 public:
8     Car(double gas): gas {gas}{}
9     void fillGas(Car c) { gas += c.gas; }
10 };
11
12 int main(int argc, const char * argv[]) {
13     Car car1 {2.0};
14     car1.fillGas(3.0);
15 }
```

1. 單參數建構函式 (例如 `Car(double gas) : gas{gas} {}`) 如果沒有用 `explicit` 關鍵字修飾，則會允許 隱式轉型。
 - 在範例中，`fillGas(Car c)` 需要一個 `Car` 型別的參數，但呼叫時使用 `fillGas(3.0)` 時，編譯器會自動將 `3.0` 轉成一個臨時的 `Car` 物件 (即 `Car(3.0)`)。
 - 因此，雖然 `fillGas()` 的參數型別是 `Car`，我們卻可以直接傳入 `double` 型態的 `3.0`，不會產生錯誤。
2. 所以第一張投影片想強調的是：
 - 如果類別的 單參數建構函式 未使用 `explicit` 修飾，就有隱式轉型的特性。
 - 這有時能帶來方便 (例如可以直接傳入數字、字串等作為參數)，但也可能讓程式變得難以閱讀，或意外發生不預期的轉型。

自动隱式轉形

explicit

```
4  class Car {
5  private:
6      double gas;
7  public:
8      explicit Car(double gas): gas {gas}{}
9      void fillGas(Car c) { gas += c.gas; }
10 };
11
12 int main(int argc, const char * argv[]) {
13     Car car1 {2.0};
14     car1.fillGas(3.0);
15 }
```

不使用隱式轉形

CE!

Compilation error

第二張投影片展示的是：

1. 將 `Car(double gas)` 前面加上了 `explicit` 關鍵字，讓它成為 `explicit Car(double gas) : gas{gas} {}`。
2. 由於建構函式變得 顯式 (`explicit`)，就不會再進行隱式轉型。
 - 這表示當函式原型是 `void fillGas(Car c)` 時，如果我們嘗試直接傳入 `3.0` (即 `fillGas(3.0)`)，編譯器就會視為錯誤，因為它找不到一個從 `double` 到 `Car` 的隱式轉型。
 - 若真的想傳入一個 `Car` 物件，那就必須手動宣告 `Car(3.0)` 或先建立 `Car car2(3.0)` 再傳入。
3. 因此，第二張投影片要表達的重點是：
 - `explicit` 關鍵字可以避免單參數建構函式自動進行隱式轉型。
 - 這對於避免意外轉型、增進程式可讀性與可維護性十分重要。

Delegating Constructor

```
4  class Rect {
5  private:
6      double length;
7      double width;
8  public:
9      Rect(double l, double w): length {l}, width {w} {}
10     Rect(double side): Rect {side, side} {}
11     void showRectSize() {
12         cout << "Length: " << length << endl;
13         cout << "Width: " << width << endl;
14     }
15 };
16
17 int main(int argc, const char * argv[]) {
18     Rect rect1 {3.0};
19     rect1.showRectSize();
20 }
```

某個建構函式可以將部分工作
委派給另一個建構函式

Default Copy Constructor

```
4  class Rect {
5  private:
6      int length;
7      int width;
8  public:
9      Rect(int l, int w): length {l}, width {w} {}
10     void showRectSize() {
11         cout << "Length: " << length << endl;
12         cout << "Width: " << width << endl;
13     }
14 };
15
16 int main(int argc, const char * argv[]) {
17     Rect rect1 {6, 8};
18     Rect rect2 = rect1;
19     Rect rect3 = Rect(rect1);
20     rect1.showRectSize(); // Length:6 Width:8
21     rect2.showRectSize(); // Length:6 Width:8
22     rect3.showRectSize(); // Length:6 Width:8
23     cout << &rect1 << ", " << &rect2 << ", " << &rect3;
24     // 0x16280      // 0x16278      // 0x16270
}
```

→ 把 rect1 copy 给 rect2

Copy Constructor

期中会考!

```
4 class Rect {
5 private:
6     int length;
7     int width;
8 public:
9     Rect(int l, int w): length {l}, width {w} {}
10    Rect(const Rect &r); ← 自行定義 Copy Constructor
11    void showRectSize() {
12        cout << "Length: " << length << endl;
13        cout << "Width: " << width << endl;
14    }
15 };
```

You can think why we use Reference!

```
17 Rect::Rect(const Rect &r) {
18     this->length = r.length + 1;
19     this->width = r.width + 2;
20 }
```

← 改寫 Copy Constructor!

```
22 int main(int argc, const char * argv[]) {
23     Rect rect1 {6, 8};
24     Rect rect2 = rect1; → 觸發 Copy Constructor
25     Rect rect3 = Rect(rect1); → 直接呼叫 Copy Constructor
26     rect1.showRectSize(); // L: 6, W: 8
27     rect2.showRectSize(); // L: 7, W: 10
28     rect3.showRectSize(); // L: 7, W: 10
29     cout << &rect1 << ", " << &rect2 << ", " << &rect3;
30         0x16280          0x16278          0x16270
}
```


Destructor

```
4 class Car {
5 public:
6     int color;
7     ~Car();
8 };
9
10 Car::~~Car() {
11     cout << this->color << endl;
12 }
13
14 int main(int argc, const char * argv[]) {
15     Car car1;
16     car1.color = 5;
17     if (true) {
18         Car car2;
19         car2.color = 3;
20     }
21 }
```

Car2 会先被
Destructor
(Link: 變數範圍!)

← Car1 在程式結束時才会被 Destructor!

輸出:

3

5

Outline

- Defining a Class
- Constructor
- Object Array
- Object and Pointer
- Friend
- Nested Class

Object Array

```
4  class Car {  
5  public:  
6      int color;  
7  };  
8  
9  int main(int argc, const char * argv[]) {  
10     Car cars[3];  
11     cars[0].color = 5;  
12     cout << cars[0].color << endl;  
13 }
```

Outline

- Defining a Class
- Constructor
- Object Array
- Object and Pointer
- Friend
- Nested Class

Pointer To Object

```
4  class Rect {
5  private:
6      int length;
7      int width;
8  public:
9      Rect(int l, int w): length {l}, width {w} {}
10     void showRectSize() {
11         cout << "Length: " << length << endl;
12         cout << "Width: " << width << endl;
13     }
14 };
15
16 int main(int argc, const char * argv[]) {
17     Rect rect1 {6, 8};
18     Rect *prect1 = &rect1;
19     prect1->showRectSize();
20 }
```

注意!

整个记忆体

只有一个物件

→ prect1 指向 rect1

Pointer Member Variable

```
4  class Rect {
5  public:
6      int length;
7      int *width;
8      void showRectSize() {
9          cout << "Length: " << length << endl;
10         cout << "Width: " << *width << endl;
11     }
12 };
13
14 int main(int argc, const char * argv[]) {
15     Rect rect1;
16     rect1.length = 6;
17     rect1.width = new int {8};
18     rect1.showRectSize();
19 }
```

Pointer Member Variable

```
4  class Rect {
5  public:
6      int length;
7      shared_ptr<int> pWidth;
8      void showRectSize() {
9          cout << "Length: " << length << endl;
10         cout << "Width: " << *pWidth << endl;
11     }
12 };
13
14 int main(int argc, const char * argv[]) {
15     Rect rect1;
16     rect1.length = 6;
17     rect1.pWidth = make_shared<int>(8);
18     rect1.showRectSize();
19 }
```

this

```
4  class Rect {
5  public:
6      int length; // for non-static member functions
7      int width;
8      void setLengthWidth(int length, int width) {
9          this->length = length;
10         this->width = width;
11     } → 谁呼叫的, this 就是谁!
12     void showRectSize() {
13         cout << "Length: " << length << endl;
14         cout << "Width: " << width << endl;
15     }
16 };
17
18 int main(int argc, const char * argv[]) {
19     Rect rect1;
20     rect1.setLengthWidth(6, 8); → this 为 rect1
21     rect1.showRectSize();
22 }
```


Method Chaining

Function Chaining

```
4 class Rect {
5 public:
6     int length;
7     int width;
8     Rect & setLength(int length) {
9         this->length = length;
10        return *this;
11    }
12    Rect & setWidth(int width) {
13        this->width = width;
14        return *this;
15    }
```

```
22 int main(int argc, const char * argv[]) {
23     Rect rect1;
24     rect1.setLength(6).setWidth(8);
25     rect1.showRectSize();
26 }
```

→ return 自己回去, 可以做 Method Chaining
(函式連鎖)

Outline

- Defining a Class
- Constructor
- Object Array
- Object and Pointer
- Friend
- Nested Class

Friend

建議非必要不要使用 (破壞資料隱藏的特性)

```
4  class Car {
5  private:
6      int color = 5;
7      friend void printColor(Car);
8  };    ↳ friend 会破壞封裝!
9
10 void printColor(Car c) {
11     cout << c.color << endl;
12 }
13
14 int main() {
15     Car car;
16     printColor(car);
17 }
```

Who can be the friend?

- General functions
- The member functions of other classes
- Other classes

Friend

```
4  class Car;
5
6  class Person {
7  public:
8      void myCar(Car c);
9  };
10
11 class Car {
12 private:
13     friend void Person::myCar(Car c);
14     int color = 5;
15 };
16
17 void Person::myCar(Car c) {
18     cout << c.color << endl;
19 }
20
21 int main() {
22     Car car;
23     Person p;
24     p.myCar(car);
25 }
```

Friend

```
4  class Car;
5
6  class Person {
7  public:
8      void myCar(Car c);
9  };
10
11 class Car {
12 private:
13     friend class Person;
14     int color = 5;
15 };
16
17 void Person::myCar(Car c) {
18     cout << c.color << endl;
19 }
20
21 int main() {
22     Car car;
23     Person p;
24     p.myCar(car);
25 }
```

Outline

- Defining a Class
- Constructor
- Object Array
- Object and Pointer
- Friend
- Nested Class

Nested Class

```
4 class Car {  
5     private: // inner class 内部類別  
6         class Driver {  
7             public:  
8                 string name;  
9         };  
10  
11         Driver driver;  
12     public:  
13         int color;  
14         void setDriverName(string name) {  
15             driver.name = name;  
16         }  
17         void showDriverName() {  
18             cout << driver.name << endl;  
19         }  
20 };  
21  
22 int main(int argc, const char * argv[]) {  
23     Car car1;  
24     car1.setDriverName("Bob");  
25     car1.showDriverName();  
26 }
```